

LoRA: Low-Rank Adaptation of Large Language Models

Authors: Rahul B, Amit S, Arnav D

<https://github.com/Arnavd543/CS-4782-LoRA>

1. Introduction

Before LoRA, the task of adapting increasingly large ML models to downstream tasks via full fine tuning was incredibly expensive as every individual parameter required gradients, optimizer states, and storage. This all changed with the paper "LoRA: Low-Rank Adaptation of Large Language Models" (Hu et al., 2021) which hypothesized that the changes made during fine tuning had a low intrinsic rank, and could therefore be approximated as a product $B \cdot A$. Then, the original weight matrix would be frozen and only these two smaller matrices are trained and then added back to the original weights (scaled by α/rank). The authors then showed that this matched full fine-tuning across the GLUE benchmark while only having to train less than 1% of parameters. Additionally, additional latency was removed by adding BA to the original weights after training.

For our project, we reimplemented LoRA from scratch on top of HuggingFace Transformers and evaluated it on SST-2 and MRPC with RoBERTa-base. We then explored four extensions of LoRA: a rank sweep across $r = 1, 2, 4, 8, 16, 32$, a target module sweep across different weight matrices in attention and FFN layers, LoRA+, which utilized asymmetric learning rates for A and B, and dropout on the LoRA path. We also ran a forgetting analysis comparing masked-LM perplexity on wikitext-2 between the pretrained model, full fine tuning, and LoRA at multiple ranks.

2. Chosen Result

For our reimplement, we decided to target the RoBERTa-base SST-2 row of Table 2 in the paper, which reported 94.8% accuracy for full fine-tuning and 95.1% accuracy for LoRA when utilizing 8 ranks for the Query and Value projections (using only 0.3M parameters out of 125M). We ultimately chose this metric as it was the paper's headline comparison with full fine-tuning and is a core result in their justification.

3. Methodology

Implementation: Our LoraLinear class is a subclass of `nn.Linear`. At construction, the original layer's weight is copied and frozen, and then two trainable parameters A (rank by `input_dim`) and B (`output_dim` by rank) are added after scaling by α/rank . We initialized A from a Gaussian distribution with mean 0 and standard deviation 0.02 and initialized B to zero, so the weight update starts at zero, and training begins from the pretrained behavior. Adapters are injected by walking `model.named_modules`, and replacing matching `nn.Linear` modules in place. Following `loralib`, we merge $B \cdot A$ into the base weight on `model.eval` and unmerge on `model.train`, giving zero latency overhead at inference time. Only LoRA parameters, the classification head, and the pooler receive gradients while all original parameters are untouched.

Setup: RoBERTa-base on SST-2 (67,349 train / 872 dev) and MRPC (3,668 train / 408 dev), max length 128, batch size 32, AdamW with linear warmup at 6 percent of steps and linear decay, and trained on Google Colab A100 GPU for 10 epochs per run. LoRA uses a learning rate $5e-4$; full fine-tuning uses $2e-5$ (same rates used in original paper). Default LoRA targets are query and value projections at rank 8 with $\alpha = 8$.

Experiments: Our first experiment was a baseline reimplement of the paper at $\text{rank}=8$, evaluated on both SST-2 and MRPC. For the second experiment we ran LoRA over 6 different ranks (1, 2, 4, 8, 16, 32) to compare performance. Additionally, for our third experiment, we tested the validation

accuracy when injecting LoRA into different matrices. In the experiment, we tested injections with rank=4 into query, key, value, QKV, attention output, FFN-up, and FFN-down weights. For our fourth experiment, we reimplemented LoRA+, utilizing asymmetric learning rates where the learning rate for B is 16 times larger than the learning rate for A. For our fifth experiment, we experimented with LoRA with dropout, randomly dropping 5 percent of input dimensions on the LoRA path (setting them to 0). Lastly, we ran a forgetting analysis comparing the masked-LM perplexity on wikitext-2 for the pretrained model, the full fine-tuned model, and LoRA at ranks 1, 8, and 32.

4. Results and Analysis

The results for our experiments are as follows:

Configuration	Trainable params	SST-2 dev acc
Full fine-tuning	124.6 M	0.9472
LoRA rank 1 (Q+V)	0.63 M	0.9392
LoRA rank 4 (Q+V)	0.74 M	0.9392
LoRA rank 8 (Q+V)	0.89 M (0.71 %)	0.9415
LoRA rank 16 (Q+V)	1.18 M	0.9427
LoRA rank 32 (Q+V)	1.77 M	0.9415
LoRA rank 4, FFN_up only	0.78 M	0.9541
LoRA rank 8 + dropout 0.05	0.89 M	0.9472
LoRA+ rank 8	0.89 M	0.9484
MRPC: Full fine-tuning	124.6 M	0.8848
MRPC: LoRA rank 8 (Q+V)	0.89 M	0.8701

Table 1. SST-2 (and MRPC) dev accuracy and trainable-parameter counts for full fine-tuning, the rank-1/4/8/16/32 sweep, the FFN_up single-module run, the dropout and LoRA+ extensions, and the MRPC baselines. RoBERTa-base, 10 epochs, batch size 32, Colab A100. Bold = best in each comparison group.

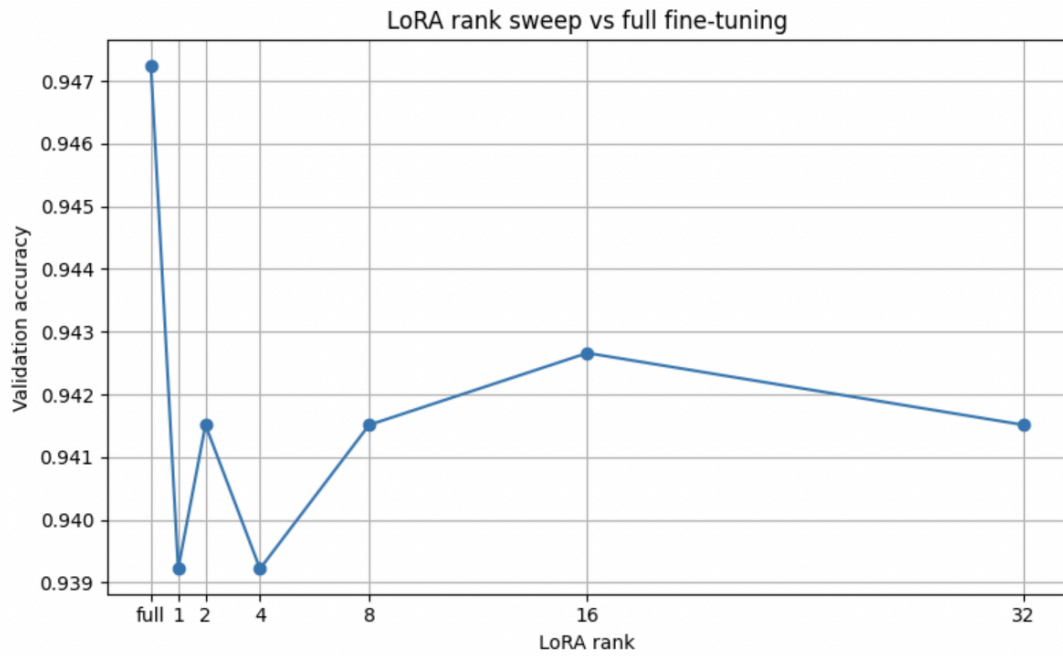


Figure 1. Rank sweep: validation accuracy vs LoRA rank on SST-2, with full fine tuning as a horizontal reference.

Our full fine tuning hit 94.72 percent on SST-2 dev and LoRA rank 8 hit 94.15 percent, which is a 0.57 point gap while training only 0.71 percent of parameters, within sampling noise of the paper's 94.8 and 95.1. Peak GPU memory dropped from 2.5 GB to 1.3 GB (a 49 percent reduction) when switching from full FT to LoRA rank 8. The MRPC baselines tell the same story (88.5 vs 87.0). We were able to reproduce the paper's headline very cleanly.

Trends from the rank sweep still hold. Accuracy is essentially flat from rank 1 (93.92) to rank 32 (94.15), with rank 16 the peak at 94.27. Rank 1, using just 0.5 percent of parameters, is only 0.23 points below rank 8. This strongly supports Hu et al.'s finding that very low intrinsic rank suffices for small classification tasks, and gains from increasing rank are within seed noise.

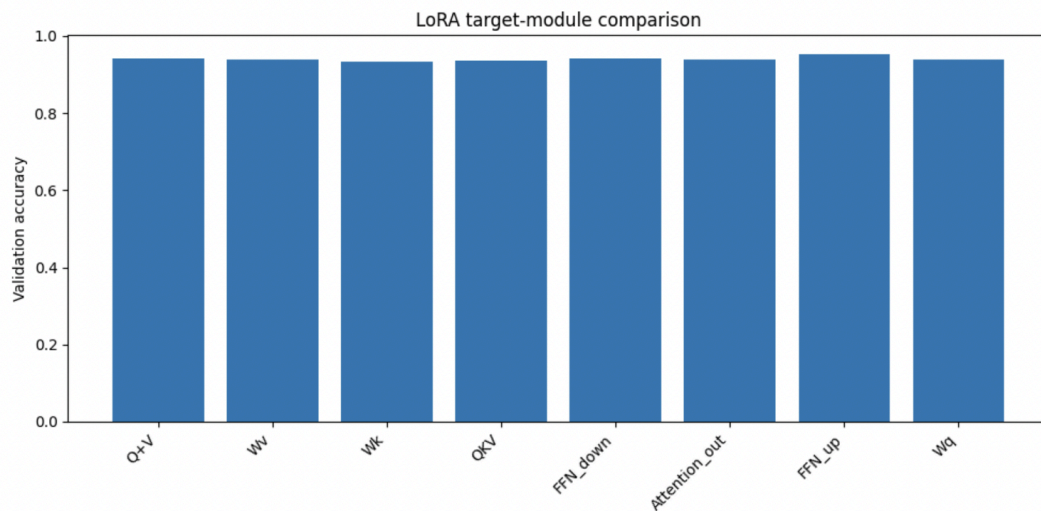


Figure 2. Module comparison: validation accuracy by LoRA target module at rank 4.

The module sweep gave us our most surprising finding. At rank=4, injecting LoRA into the FFN up-projection (intermediate.dense) alone hit 95.41 percent, beating both Q+V (the paper's default choice) and full fine tuning. Within attention, V outperformed Q which outperformed K (the worst single target at 93.35), matching the ordering reported in the paper.

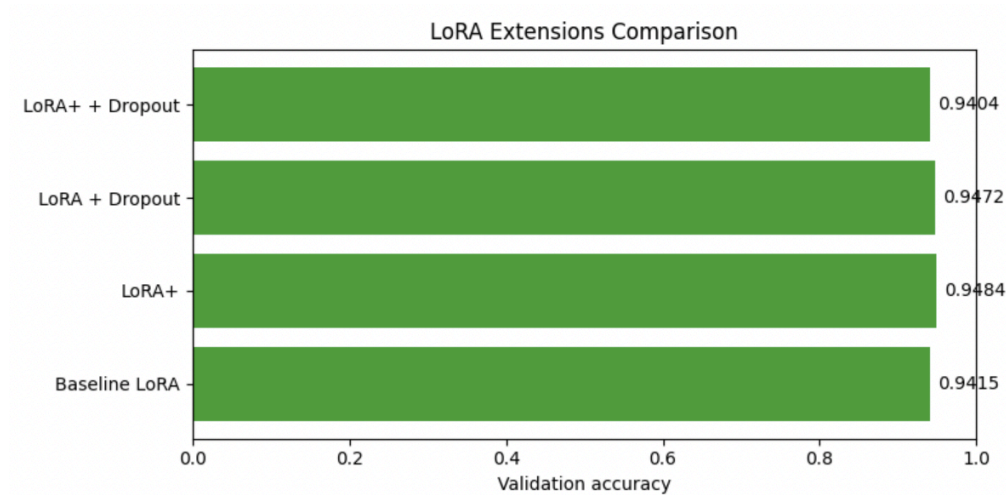


Figure 3. Extensions comparison: baseline LoRA vs LoRA+ vs LoRA + Dropout vs LoRA+ + Dropout.

LoRA+ was the strongest of our extensions. With lr_B set to 16 times lr_A, LoRA+ reached 94.84 percent, beating both baseline LoRA and full fine tuning. Adding dropout 0.05 to baseline LoRA also helped marginally (94.72 vs 94.15), but combining LoRA+ with dropout dropped to 94.04 percent, potentially from over-regularization on the small dataset.

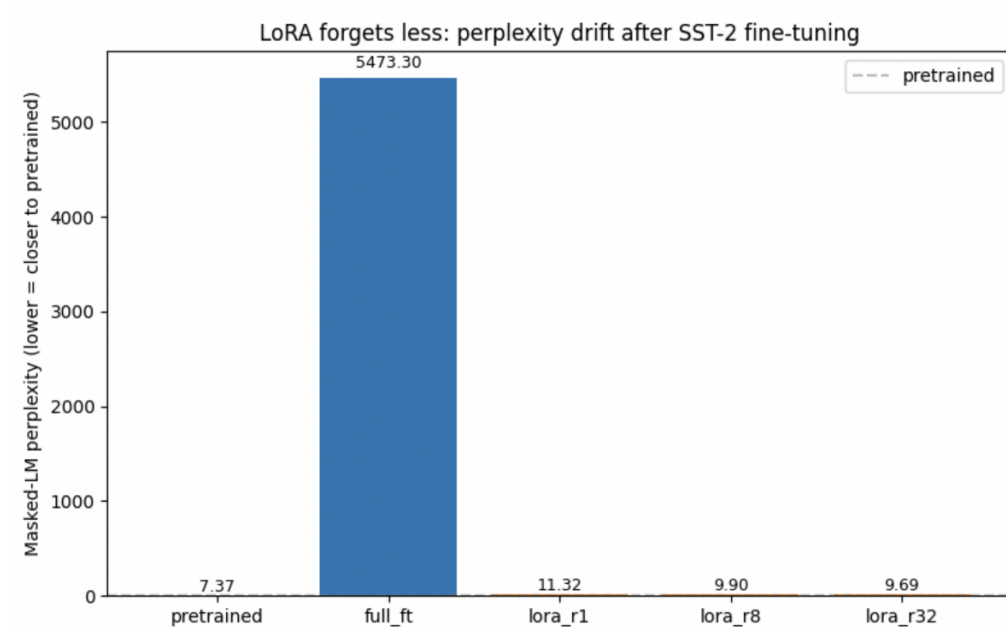


Figure 4. Masked-LM perplexity drift on wikitext-2 after SST-2 fine tuning.

The forgetting analysis showed that LoRA helps maintain more of the model’s original language capabilities compared to full fine tuning. The pretrained model has wikitext-2 masked-LM perplexity 7.37. After full SST-2 fine tuning, perplexity increases dramatically to 5,473, roughly 740 times the pretrained value. On the other hand, every LoRA variant stays close to the pretrained baseline, with $r=1$ at 11.32 (+54%), $r=8$ at 9.90 (+34%), $r=32$ at 9.69 (+31%), which indicates the LoRA fine tuning helps us retain more general language capabilities. A secondary finding here is that with alpha held fixed at 8, higher ranks forget less. Before experimentation, a sensible prediction would be that giving more capacity to the fine tuning mechanism would cause more drift from the original model’s capabilities, as the model became more specialized to the new task. However, $r=1$ applies B^*A at scaling 8, $r=8$ at scaling 1, and $r=32$ at scaling 0.25, so the effective per-step weight update is largest at the lowest rank. So, the magnitude of the weight update by changing alpha can protect LoRA from forgetting, not just the rank itself.

5. Reflections

Implementing LoRA from scratch showed us that the merge/unmerge mechanism is one of the most important practical aspects of LoRA’s methods. Merging BA into the frozen pretrained weights at inference time removes additional latency and makes LoRA a true drop-in replacement for full fine tuning. We also found that reported “trainable parameter” counts can be misleading, as the paper’s 0.3M figure for rank-8 LoRA excludes the classification head, which must still be trained from scratch and adds roughly another 0.6M parameters.

Our most surprising result came from the module sweep. Injecting LoRA into the FFN up-projection alone at rank 4 slightly outperformed both attention-only LoRA and full fine tuning on SST-2, suggesting that the feedforward network may be at least as important a target for adaptation as the query and value projections. This is likely task-dependent, though. The forgetting analysis also showed that LoRA preserves pretrained language capabilities far better than full fine tuning, and that with alpha fixed, higher ranks forgot slightly less because the effective update magnitude scales with alpha over r .

If extending and improving this work, we would run multiple seeds to establish confidence intervals across the rank sweep, as the seed is a cause of variance, perform a focused alpha sweep to isolate the scaling effect on forgetting, and implement AdaLoRA (Zhang et al., 2023) as a next step for adaptive rank allocation. In short, this project reinforced the central idea behind parameter-efficient fine-tuning. With a small reduction in expressivity, we can get major gains in memory usage, checkpoint size, training cost, and deployment practicality.

References

- [1] Hu, E. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
- [2] Hayou, S., Ghosh, N., Yu, B. (2024). LoRA+: Efficient Low Rank Adaptation of Large Models. arXiv:2402.12354.
- [3] Zhang, Q. et al. (2023). AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. arXiv:2303.10512.
- [4] Liu, Y. et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.
- [5] Wang, A. et al. (2018). GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. arXiv:1804.07461.